

Компилятор метаданных сервиса

Для ускорения запуска автономных приложений в платформе была создана утилита **компилятор ресурсов**. Эта утилита превращает файлы метаданных сервиса в бинарный код, который компилируется вместе с программой, чтобы при запуске приложения не требовалось искать эти файлы на диске и читать их. Данная утилита работает в плагинной системе и поддерживает расширяемый набор компиляторов.

На данный момент поддерживаются следующие виды ресурсов:

Ключ для RESOURCES_LIST	Обрабатываемые метаданные
dicx	Структура БД, описанная в файлах .dicx
orx	Файлы .orx с описаниями методов БЛ.
lang	Файлы перевода.
bin_lang	В отличие от lang создает из ресурсов нативную структуру.
res_pack	Дополнительные ресурсы БД, описанные в файлах .cnv и .drack
static_link_order	Служебный компилятор для порядка статической линковки.
srv	Файлы s3srv и s3mod.
export_navx	Компилятор бинарного представления файлов navx.
hox	Компилятор бинарного представления файлов словарей переводов.
feature	Компилятор бинарного представления файлов feature.
light_right	Компилятор бинарного представления файлов упрощенной системы прав.
navigation_items	Компилятор для навигации для items.
config_info	Компилятор для составления справочника по файлам cfg.
psk	Компилятор бинарного представления файлов psk.
rights	Компилятор бинарного представления файлов uax, rlx.
wi_tables	Компилятор таблиц по микросервисам для скриптов автодокументации.

Создание своего компилятора метаданных

Чтобы создать свой компилятор метаданных, выполните следующие действия:

1. Создайте cpp-файл, в котором будет располагаться компилятор в [nanke](#) и добавить его в [CMakeLists.txt](#).
2. Создайте класс своего контроллера, унаследованный от **IResourcesCompiler**:

```
class NewCompiler: public IResourcesCompiler;
```

3. Реализуйте функции:

- IResourcesCompiler::Name(), возвращающую имя компилятора. Имя не должно содержать пробелы.

```
1 StringView NewCompiler::Name()  
2 {  
3     return L"new_compiler"_sv;
```

4 } }

- `IResourcesCompiler::OutputFile()`, содержащую имя выходного файла.
- `IResourcesCompiler::Group()`. Список файлов, относящихся к одной группе, будет выведен в файл с соответствующим названием для последующей обработки в **CMake**. На данный момент, если компилятор создаёт сср-файл, то группа должна называться "sources", прочие группы не обрабатываются, можно возвращать `std::nullopt`.
- `IResourcesCompiler::InputFiles()`. Эта функция возвращает список файлов ресурсов, обрабатываемых данным компилятором, чтобы при сборке перезапустить компилятор, если они изменились. Для добавления всех файлов сервиса можно использовать следующую реализацию:
- `IResourcesCompiler::Compile()`. Это основная функция, в которой происходит создание выходного файла. Данные можно записывать в аллокатор через **StringBuilder**:

```
1 void NewCompiler::Compile( ld::Service const& srv, Allocator<
  wchar_t > const& output_alloc,
2                               const String& sdk_version,
  ld::PlatformConfig const& config )
3 {
4     auto sb = MakeStringBuilder( output_alloc );
5     sb << L"some data";
6 }
```

4. Зарегистрируйте свой компилятор, создав статическую переменную регистратора:

```
static ResourcesCompilerRegistrar< NewCompiler > new_compiler_reg;
```

где **NewCompiler** - имя класса вашего компилятора.

5. Подключите новый компилятор в **CMakeLists.txt** прикладного проекта в вызове `sbis_create_project`:

```
1 sbis_create_project( our_project library SERVICE_PATH
  ../project_location/our_project.s3srv
2                               RESOURCES_LIST new_compiler
  )
```

где `../project_location/our_project.s3srv` - относительный путь к файлу сервиса, для которого надо запустить компилятор, а в `RESOURCES_LIST` перечисляются все компиляторы, которые надо запустить в соответствии с их именами, возвращаемыми из `IResourcesCompiler::Name()`.

Оффлайн-приложения построены по сервисной архитектуре, в которой есть координатор и несколько рабочих процессов (пулов), исполняющих задачи. Пулы и координатор можно бинаризировать по отдельности. Чтобы скомпилировать ресурсы для координатора, нужно создать две библиотеки:

- Сам координатор и к нему добавить его `srv`-файл.
- Библиотеку, поставляющую структуру проекта.

Для бинаризации отдельного пула необходимо создать библиотеку, в которую запускаются все компиляторы. Эта библиотека оформляется в [структуре проекта приложения](#). Если вы бинаризировали в вашем приложении и то, и другое, то все ресурсы, которые подверглись компиляции можно удалить из итогового дистрибутива.